

Playing Atari with Deep Reinforcement Learning

B.Sc. Seminar Decision Making Algorithms

Technische Universität München
TUM School of Computation, Information and Technology
Decision Sciences & Systems
Department of Computer Science

Betreut von Prof. Dr. Jalal Etesami
Decision Sciences & Systems
Department of Computer Science

Eingereicht von Sebastian Rogg
Luca Bozzetti
Paul Vorderbrügge

Eingereicht am München, den 16.07.2025

Motivation and Introduction

In 2013, Mnih et al. introduced the Deep Q-Network (DQN), the first successful combination of deep learning and reinforcement learning that learns control policies directly from raw pixel inputs. The motivation was to create a system that could learn end-to-end without hand-crafted features or prior domain knowledge. Traditional reinforcement learning methods relied on low-dimensional, hand-engineered state representations. In contrast, DQN learned directly from high-dimensional inputs (frames), mimicking how humans play video games using only visual feedback.¹

The Atari environment is formalized as a Markov Decision Process. At each time step t , the agent observes a state s_t , takes an action a_t from a discrete set, and potentially receives a reward r_t . The goal is to maximize the expected reward:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where $\gamma \in [0, 1]$ is the discount factor.

DQN builds upon the Q-learning algorithm, which iteratively updates Q-values using the Bellman equation to approximate a target y :

$$y = r + \gamma \max_{a'} Q(s', a')$$

Here, the target approximates the true Q-value by bootstrapping from the current estimate of the next state's value. Through this, we converge to the optimal action-value function Q^* , as the learned Q-values Q increasingly approximate Q^* with sufficient training and exploration.²

Deep Q-Network Algorithm

The agent learns a Q-function $Q(s, a)$ that estimates the expected future reward for taking action a in state s . The state consists of four consecutive frames stacked together to capture motion information. A tabular Q-function is infeasible due to the large input space, so DQN approximates $Q(s, a; \theta)$ using a convolutional neural network (CNN) with parameters θ . The network outputs Q-values for all actions a given the current state s . The states are pre-processed into a lower-dimensional representation $\phi(s)$ (e.g., converting to gray-scale and down-sampling from 210×160 to 84×84 pixels) before being passed to the network.

An ϵ -greedy strategy is used to balance exploration and exploitation: with probability ϵ , a random action is selected; otherwise, the action with the highest Q-value is taken. ϵ is reduced during training to shift from exploration to exploitation.

Instead of learning from sequential experiences, which are temporally correlated, DQN stores transitions

¹ Mnih; Kavukcuoglu; Silver; Graves, et al., 2013.

² Sutton; Barto, 1998.

(s_t, a_t, r_t, s_{t+1}) in a replay memory.³ During the inner loop, random batches are sampled from this memory, improving data efficiency and stability.⁴ These batches are used to compute a loss between predicted and target Q-values, and through gradient descent, the weights θ of the CNN are updated to minimize this loss. The full DQN algorithm is presented in Algorithm 1.

Algorithm 1 Deep Q-learning with Experience Replay⁵

```

1: Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights
3: for episode = 1,  $M$  do
4:   Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
5:   for  $t = 1, T$  do
6:     With probability  $\epsilon$  select a random action  $a_t$ 
7:     otherwise select  $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$ 
8:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
9:     Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
10:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
11:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
12:    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
13:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$ 
14:  end for
15: end for

```

The architecture used processes $84 \times 84 \times 4$ preprocessed frames through two convolutional layers using ReLU activations. This is followed by a fully-connected layer with 256 ReLU units and a final linear output layer with one unit per valid action (4-18 actions depending on the game).⁶

Training Setup and Results

To reproduce the results from Mnih et al.,⁷ we used the Arcade Learning Environment (ALE), which provides a convenient Python interface for interacting with Atari games. Our implementation was based on PyTorch and used the hyperparameters specified in their later work.⁸ Training was conducted for approximately 8 hours on a consumer-grade GPU. The resulting agent achieved an average score of 20, which is nearly optimal given that the maximum achievable score in the environment is 21. Our full implementation and detailed results can be found on GitHub: DQN_Atari2600.

As shown in Table 1, DQN achieves better than human-level performance in games like Pong or Breakout. Overall DQN surpassed all previous methods on 6 out of 7 tested games,⁹

³ Lin, 1993.

⁴ Sutton; Barto, 1998.

⁶ Mnih; Kavukcuoglu; Silver; Graves, et al., 2013.

⁷ Mnih; Kavukcuoglu; Silver; Graves, et al., 2013.

⁸ Mnih; Kavukcuoglu; Silver; Rusu, et al., 2015.

⁹ Mnih; Kavukcuoglu; Silver; Graves, et al., 2013.

Table 1 Performance comparison across Atari games¹⁰

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	−3	18900	28010	3690

Conclusion

DQN represents a breakthrough in Reinforcement Learning, showing for the first time that agents can learn control policies directly from raw pixel input using deep reinforcement learning. It achieved strong performance across many Atari games, highlighting the power of end-to-end learning without handcrafted features. Despite its success, DQN has clear limitations. It takes very long to train, struggles with sparse reward environments like Montezuma’s Revenge, and is prone to instability and sensitivity to hyperparameters.

These issues have been addressed in follow-up work: Double DQN¹¹ reduces overestimation bias by decoupling action selection and action evaluation during target computation. Dueling DQN¹² improves value estimation by separately learning the state-value function and the advantage function. DQN laid the foundation for all these developments and paved the way for advanced systems like AlphaGo and MuZero, which combine deep learning and planning to achieve superhuman performance in complex games like Chess or Go.

¹¹Hasselt; Guez; Silver, 2015.

¹²Wang et al., 2016.

References

- HASSELT, Hado van; GUEZ, Arthur; SILVER, David, 2015. *Deep Reinforcement Learning with Double Q-learning*. Available from arXiv: 1509.06461 [cs.LG].
- LIN, Long-Ji, 1993. *Reinforcement learning for robots using neural networks*. Technical report. DTIC Document.
- MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David; GRAVES, Alex; ANTONOGLOU, Ioannis; WIERSTRA, Daan; RIEDMILLER, Martin, 2013. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*. Available also from: <https://arxiv.org/abs/1312.5602>.
- MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David; RUSU, Andrei A; VENESS, Joel; BELLE-MARE, Marc G; GRAVES, Alex; RIEDMILLER, Martin; FIDJELAND, Andreas K; OSTROVSKI, Georg; PETERSEN, Stig; BEATTIE, Charles; SADIK, Amir; ANTONOGLOU, Ioannis; KING, Helen; KUMARAN, Dharshan; WIERSTRA, Daan; LEGG, Shane; HASSABIS, Demis, 2015. Human-level control through deep reinforcement learning. *Nature*. Volume 518, number 7540, pp. 529–533. Available from DOI: 10.1038/nature14236.
- SUTTON, Richard S.; BARTO, Andrew G., 1998. *Reinforcement Learning: An Introduction*. MIT Press.
- WANG, Ziyu; SCHAUL, Tom; HESSEL, Matteo; HASSELT, Hado van; LANCTOT, Marc; FREITAS, Nando de, 2016. *Dueling Network Architectures for Deep Reinforcement Learning*. Available from arXiv: 1511.06581 [cs.LG].